

Week 3.

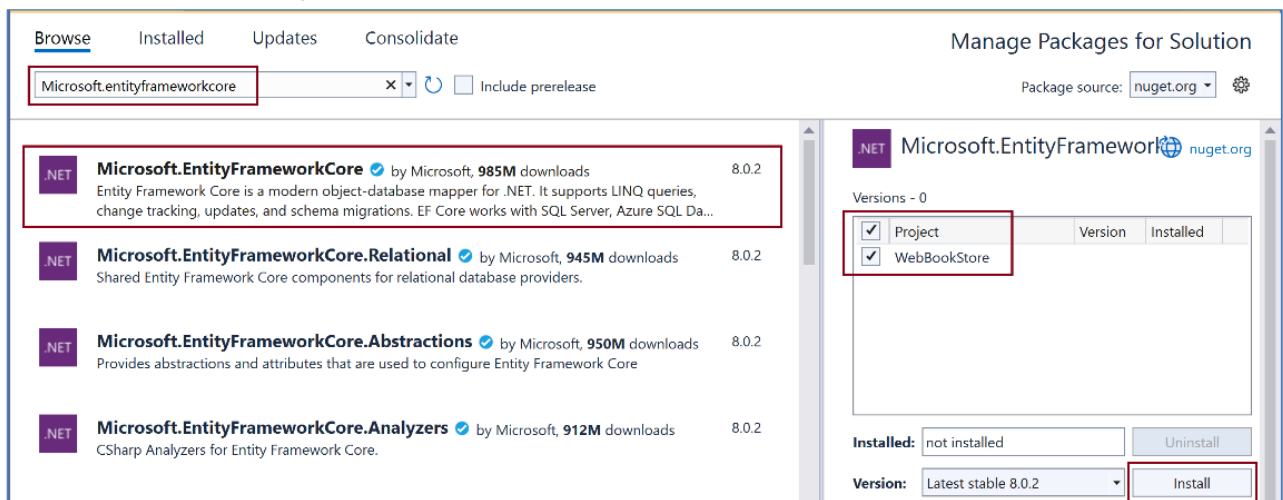
ASP.NET CORE APPLICATION (P1)

ENTITY FRAMEWORK CORE

(CODE FIRST)

INSTALL NUGET PACKAGE

- Mở project **WebBookStore**
- Tool > NuGet Package Manager > Manager NuGet Packages for Solution ...
- Cài đặt thêm:
 - + **Microsoft.EntityFrameworkCore**



- + **Microsoft.EntityFrameworkCore.SqlServer**
- + **Microsoft.EntityFrameworkCore.Tools**

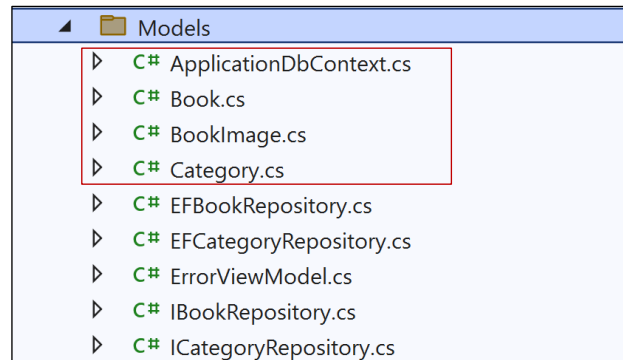
- Kích hoạt NuGet:

```
Command Prompt
C:\Users\trinh>dotnet tool install --global dotnet-ef --version 8.*
Skipping NuGet package signature verification.
```

dotnet tool install --global dotnet-ef --version 8.*

Models

Cấu trúc thư mục tại Models



1/ Tạo class **Book.cs**

```
5 public class Book
6 {
7     11 references
8     public int ID { get; set; }
9     [Required,StringLength(50)]
10    11 references
11    public string Title { get; set; }
12    11 references
13    public string Author { get; set; }
14    11 references
15    public int PublishYear { get; set; }
16    11 references
17    public double Price { get; set; }
18    13 references
19    public string Cover { get; set; }
20    0 references
21    public List<BookImage>? Images {get;set;}
22    1 reference
23    public int CategoryId { get; set; }
24    3 references
25    public Category? Category { get; set; }
26 }
27 }
```

2/ Tạo class **Category.cs**

```
5 public class Category
6 {
7     0 references
8     public int Id { get; set; }
9     [Required,StringLength(50)]
10    1 reference
11    public string Name { get; set; }
12    0 references
13    public List<Book> Books { get; set; }
14 }
```

3/ Tạo class BookImage.cs

```
3 public class BookImage
4 {
5     0 references
    public string Id { get; set; }
6     0 references
    public string Url { get; set; }
7     0 references
    public int BookId { get; set; }
8     0 references
    public Book? Book { get; set; }
9 }
10 }
```

Cấu hình Entity Framework Core

#1. Tạo lớp ApplicationDbContext.cs trong Models

```
using Microsoft.EntityFrameworkCore;

namespace WebBookStore.Models
{
    9 references
    public class ApplicationDbContext: DbContext
    {
        0 references
        public ApplicationDbContext(DbContextOptions<ApplicationDbContext> options) : base(options) { }
        6 references
        public DbSet<Book> Books { get; set; }
        6 references
        public DbSet<Category> Categories { get; set; }
        0 references
        public DbSet<BookImage> BookImages { get; set; }
    }
}
```

#2. Cấu hình EF Core trong ‘Program.cs’

```
1 using Microsoft.EntityFrameworkCore;
2 using WebBookStore.Models;
3
4 var builder = WebApplication.CreateBuilder(args);
5
6 builder.Services.AddDbContext<ApplicationDbContext>
7     (options => options.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection")));
8
9 // Add services to the container.
10 builder.Services.AddControllersWithViews();
```

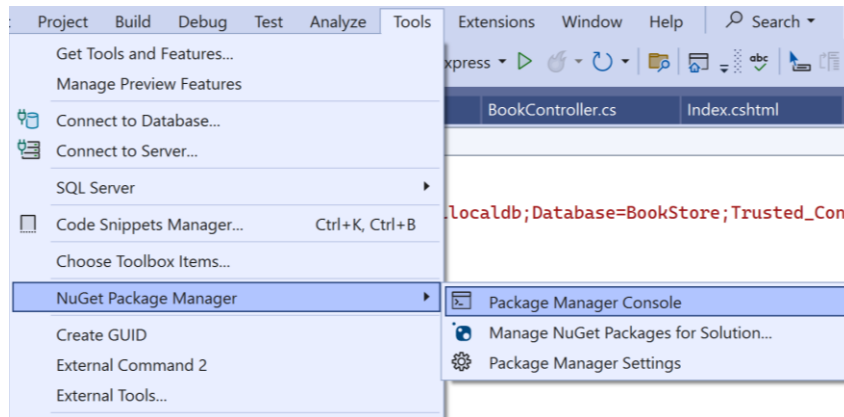
#3. Cấu hình connection string trong ‘appsettings.json’

```
1 {
2   "ConnectionStrings": {
3     "DefaultConnection": "Server=(localdb)\\mssqllocaldb;Database=BookStore;Trusted_Connection=True;MultipleActiveResultSets=true"
4   },
5   "Logging": {
6     "LogLevel": {
7       "Default": "Information",
8       "Microsoft.AspNetCore": "Warning"
9     }
10  }
```

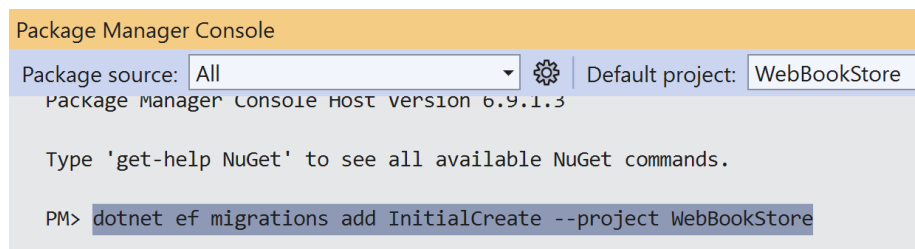
```
"ConnectionStrings": {
  "DefaultConnection": "Server=(localdb)\\mssqllocaldb;Database=BookStore;Trusted_Connection=True;MultipleActiveResultSets=true"},
```

#4. Tạo Migrations

- Click phải tên project > **Build**
- Tools > NuGet Package Manager > Package Manager Console

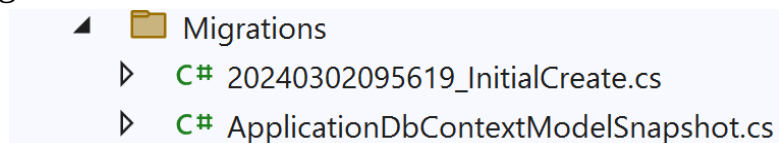


- Tạo Migrations (để tạo cơ sở dữ liệu)

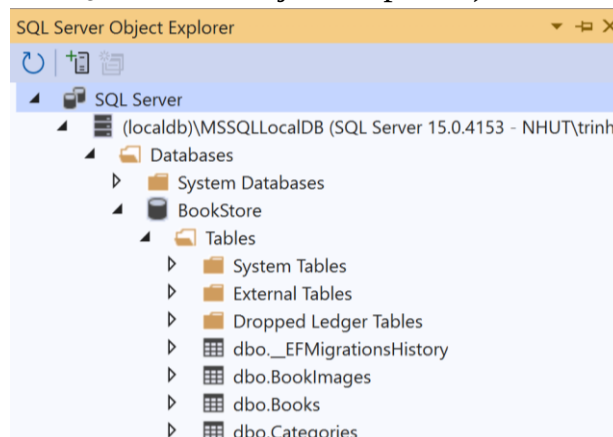


`dotnet ef migrations add InitialCreate --project WebBookStore`
`dotnet ef database update --project WebBookStore`

- Kết quả thực hiện:
 - + Thư mục Migrations



- + Cơ sở dữ liệu (View/SQL Server Object Explorer)



+ Nhập dữ liệu cho các bảng:

dbo.Categories [Data] appsettings.json*		
	Id	Name
▶	1	IT
	2	Economic
	3	Literature
*	NULL	NULL

dbo.Books [Data] dbo.Categories [Data] appsettings.json* BookController.cs							
	ID	Title	Author	PublishYear	Price	Cover	CategoryId
▶	3	Title 2	Author 2	2023	10	images/book2.jpg	2
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Định nghĩa các interface chứa các phương thức cần thiết để thao tác dữ liệu

4/ IBookRepository.cs

```

1  using System.Collections.Generic;
2  using WebBookStore.Models;
3  namespace WebBookStore.Models
4  {
5      4 references
6      public interface IBookRepository
7      {
8          2 references
9          Task<IEnumerable<Book>> GetAllAsync();
10         4 references
11         Task<Book> GetByIdAsync(int id);
12         2 references
13         Task AddAsync(Book book);
14         2 references
15         Task UpdateAsync(Book book);
16         2 references
17         Task DeleteAsync(int id);
18     }
19 }

```

5/ ICategoryRepository.cs

```

1  namespace WebBookStore.Models
2  {
3      4 references
4      public interface ICategoryRepository
5      {
6          4 references
7          Task <IEnumerable<Category>> GetAllAsync ();
8          1 reference
9          Task<Category> GetByIdAsync (int id);
10         1 reference
11         Task AddAsync(Category category);
12         1 reference
13         Task UpdateAsync(Category category);
14         1 reference
15         Task DeleteAsync (int id);
16     }
17 }

```


6/ EFBookRepository.cs

```
1  using Microsoft.EntityFrameworkCore;
2
3  namespace WebBookStore.Models
4  {
5      2 references
6      public class EFBookRepository: IBookRepository
7      {
8          private readonly ApplicationDbContext _context;
9          0 references
10         public EFBookRepository(ApplicationDbContext context)
11         {
12             _context = context;
13         }
14         2 references
15         public async Task<IEnumerable<Book>> GetAllAsync()
16         {
17             return await _context.Books.ToListAsync();
18         }
19         4 references
20         public async Task<Book> GetByIdAsync(int id)
21         {
22             return await _context.Books.FindAsync(id);
23         }
24         2 references
25         public async Task AddAsync(Book book)
26         {
27             _context.Books.Add(book);
28             await _context.SaveChangesAsync();
29         }
30         2 references
31         public async Task UpdateAsync(Book book)
32         {
33             _context.Books.Update(book);
34             await _context.SaveChangesAsync();
35         }
36         2 references
37         public async Task DeleteAsync(int id)
38         {
39             var book = await _context.Books.FindAsync(id);
40             _context.Books.Remove(book);
41             await _context.SaveChangesAsync();
42         }
43     }
44 }
```

7/ EFCategoryRepository.cs

```
1  using Microsoft.EntityFrameworkCore;
2
3  namespace WebBookStore.Models
4  {
5      2 references
6      public class EFCategoryRepository: ICategoryRepository
7      {
8          private readonly ApplicationDbContext _context;
9          0 references
10         public EFCategoryRepository(ApplicationDbContext context)
11         {
12             _context = context;
13         }
14         4 references
15         public async Task<IEnumerable<Category>> GetAllAsync()
16         {
17             return await _context.Categories.ToListAsync();
18         }
19         1 reference
20         public async Task<Category> GetByIdAsync(int id)
21         {
22             return await _context.Categories.FindAsync(id);
23         }
24         1 reference
25         public async Task AddAsync(Category category)
26         {
27             _context.Categories.Add(category);
28             await _context.SaveChangesAsync();
29         }
30         1 reference
31         public async Task UpdateAsync(Category category)
32         {
33             _context.Categories.Update(category);
34             await _context.SaveChangesAsync();
35         }
36         1 reference
37         public async Task DeleteAsync(int id)
38         {
39             var category = await _context.Categories.FindAsync(id);
40             _context.Categories.Remove(category);
41             await _context.SaveChangesAsync();
42         }
43     }
44 }
```

8/ Đăng ký interface trong Program.cs

```
9  // Add services to the container.
10 builder.Services.AddControllersWithViews();
11
12 builder.Services.AddScoped<IBookRepository, EFBookRepository>();
13 builder.Services.AddScoped<ICategoryRepository, EFCategoryRepository>();
```


Controllers

Hiệu chỉnh **BookController.cs** trong Controllers

```
1  using Microsoft.AspNetCore.Mvc;
2  using Microsoft.AspNetCore.Mvc.Rendering;
3  using WebBookStore.Models;
4
5  namespace WebBookStore.Controllers
6  {
7      1 reference
8      public class BookController : Controller
9      {
10         private readonly IBookRepository _bookRepository;
11         private readonly ICategoryRepository _categoryRepository;
12         0 references
13         public BookController(IBookRepository bookRepository, ICategoryRepository categoryRepository)
14         {
15             _bookRepository = bookRepository;
16             _categoryRepository = categoryRepository;
17         }
18         //Show all books
19         2 references
20         public async Task<IActionResult> Index()
21         {
22             var books = await _bookRepository.GetAllAsync();
23             return View(books);
24         }
25         //Show add new book page
26         0 references
27         public async Task<IActionResult> Add()
28         {
29             var categories = await _categoryRepository.GetAllAsync();
30             ViewBag.Categories = new SelectList(categories, "Id", "Name");
31             return View();
32         }
33         [HttpPost]
34         0 references
35         public async Task<IActionResult> Add(Book book)
36         {
37             if(ModelState.IsValid)
38             {
39                 await _bookRepository.AddAsync(book);
40                 return RedirectToAction("Index");
41             }
42             var categories = await _categoryRepository.GetAllAsync();
43             ViewBag.Category = new SelectList(categories, "Id", "Name");
44             return View(book);
45         }
46         //Show detail of a book
47         0 references
48         public async Task<ActionResult> Detail(int id)
49         {
50             var book = _bookRepository.GetByIdAsync(id);
51             if(book==null)
52             {
53                 return NotFound();
54             }
55             return View(book);
56         }
57         //Show update page of the book
58         0 references
```

```

52  ✓ public async Task<IActionResult> Update(int id)
53  {
54      var book =await _bookRepository.GetByIdAsync(id);
55      if(book==null)
56      {
57          return NotFound();
58      }
59      var categories = await _categoryRepository.GetAllAsync();
60      ViewBag.Categories = new SelectList(categories, "Id", "Name",book.CategoryId);
61      return View(book);
62  }
63  //Process the book update
64  [HttpPost]
65  0 references
66  public async Task<IActionResult> Update(int id, Book book)
67  {
68      if(id != book.ID)
69      {
70          return NotFound();
71      }
72      if(ModelState.IsValid)
73      {
74          await _bookRepository.UpdateAsync(book);
75          return RedirectToAction(nameof(Index));
76      }
77      return View(book);
78  }
79  //Show the book delete confirmation
80  0 references
81  public async Task<IActionResult> Delete(int id)
82  {
83      var book = await _bookRepository.GetByIdAsync(id);
84      if(book==null)
85      {
86          return NotFound();
87      }
88      return View(book);
89  }
90  //Process the book deletion
91  [HttpPost,ActionName("Delete")]
92  0 references
93  public async Task<IActionResult> DeleteConfirmed(int id)
94  {
95      await _bookRepository.DeleteAsync(id);
96      return RedirectToAction(nameof(Index));
97  }

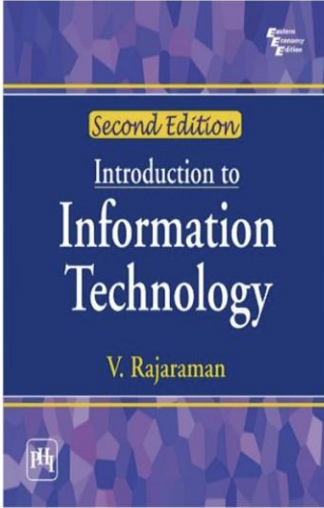
```

View

Trang Index page (layout):

```
1  @using System.Collections;
2  @model IEnumerable<WebBookStore.Models.Book>
3  @{
4      ViewData["Title"] = "Index";
5      Layout = "~/Views/Shared/_Layout.cshtml";
6  }
7  <h1>Index</h1>
8  <a asp-controller="Book" asp-action="Add">Add a book</a>
9  <table class="table">
10     <thead>
11     <tr>
12         <th>Title</th>
13         <th>Author</th>
14         <th>PublishYear</th>
15         <th>Price</th>
16         <th>Category</th>
17         <th>Cover</th>
18         <th></th>
19     </tr>
20     </thead>
21     <tbody>
22         @foreach(var item in Model)
23         {
24             <tr>
25                 <td>@item.Title</td>
26                 <td>@item.Author</td>
27                 <td>@item.PublishYear</td>
28                 <td>@item.Price</td>
29                 <td>@item.CategoryId</td>
30                 <td></td>
31                 <td>
32                     <a asp-action="Detail" asp-route-id="@item.ID">View</a>
33                     <a asp-action="Update" asp-route-id="@item.ID">Update</a>
34                     <a asp-action="Delete" asp-route-id="@item.ID">Delete</a>
35                 </td>
36             </tr>
37         }
38     </tbody>
39 </table>
```

Xem kết quả trang Index

Index					
Add a book					
Title	Author	PublishYear	Price	Category	Cover
Title 2	Author 2	2023	10	2	
					View Update Delete

(*) Yêu cầu sinh viên tự thực hiện:

1/ Thực hiện các thao tác Add, View, Update, Delete trên trang Index của Book.

2/ Tạo trang **Index của Category** với các thao tác tương tự như trang **Index của Book**.